# 🎬 DevLog — Loom Video Script (≤ 4 min)

> **Total target time:** 3 min 45 sec
> **Format:** screen recording with face cam
> **Tabs to have open before recording:** localhost:3000, VS Code (files: validations.ts, api/entries/route.ts, log/page.tsx)

---

## PART 1 — App Intro & Feature Demo (0:00 – 1:30)

### 0:00 – 0:12 | Hook

> *[Show dashboard]*

"This is DevLog — a personal learning journal for developers. Every entry, project, and resource you've worked on, in one place. Let me walk you through it."

---

### 0:12 – 0:30 | Dashboard

> *[Point to streak counter, weekly chart, and recent entries]*

"The dashboard shows your current writing streak, a weekly activity chart, and quick links to your latest entries and projects. All data is live from PostgreSQL via React Query — no page reloads needed."

---

### 0:30 – 0:55 | Log Entries

> *[Navigate to /log, click "New Log Entry"]*

"These are log entries — daily development notes written in Markdown. I'll create one now."

> *[Fill in: Title = "Adding Zod validation", Date = today, Body = "Today I added schema validation to all API routes...", tag = "backend", link a project]*

"Notice the Markdown body, tag input, and the ability to link an existing Project or create a new one inline — all from this one form."

> *[Submit it — toast appears, new card appears in the list]*

"The new entry appears instantly. React Query invalidates the cache on success, so the list updates without a full reload."

---

### 0:55 – 1:15 | Projects

> *[Navigate to /projects, open a project detail page]*

"Projects group your work. Each project shows its linked log entries and resources in the sidebar. You can also link entries and resources directly from the project edit form."

---

### 1:15 – 1:30 | Resources

> *[Navigate to /resources, show category filters, toggle Favourite/Read]*

"Resources is a bookmark manager. You can filter by category — Article, Video, Docs — and by read/favourite status. Clicking the bookmark icon marks it as read in a single PATCH call."

---

## PART 2 — Code Walkthrough: POST /api/entries (1:30 – 2:45)

> *[Switch to VS Code, open [src/lib/validations.ts](src/lib/validations.ts)]*

### 1:30 – 1:50 | Zod Schema

"Let's look at how an entry gets validated. Here's the `CreateEntrySchema` in validations.ts."

> *[Highlight lines 15–33]*

```
export const CreateEntrySchema = z.object({
  title: z.string().min(1).max(100).trim(),
  date:  z.string().min(1),
  body:  z.string().min(1),
  tags:  z.array(z.string()).min(0),
  projectIds:  z.array(z.string()).min(0),
  resourceIds: z.array(z.string()).min(0).optional(),
});
```

"Zod gives us runtime type safety. `title` must be a non-empty string under 100 chars, `tags` and `projectIds` are arrays of IDs. `UpdateEntrySchema` just extends this with a required `id`

field."

---

## 1:50 – 2:20 | API Route

> *[Open src/app/api/entries/route.ts]*

"The POST handler calls `CreateEntrySchema.safeParse(body)` — this is non-throwing, so we can check `validated.success` and return a structured 400 if anything fails."

> *[Scroll to lines 66–93]*

"If validation passes, we upsert tags first — creating any tag that doesn't exist yet — then create the entry and connect its junction table rows for projects and resources in a single Prisma transaction. The response flattens the junction tables back into plain arrays so the frontend doesn't have to."

---

## 2:20 – 2:45 | React Query Consumption

> *[Open src/app/log/page.tsx, show lines 25–46]*

"On the frontend, `useQuery` with key `["entries"]` fetches the list. When EntryForm submits successfully, its `useMutation` calls `queryClient.invalidateQueries({ queryKey: ["entries"] })` — that's what causes the list to re-render immediately with the new entry. No manual state update needed."

---

# PART 3 — Technical Challenge (2:45 – 3:35)

> *[Switch back to browser or open ResourceForm.tsx]*

## 2:45 – 3:05 | The Problem

"The biggest architectural challenge was a **circular dependency between the three forms**. EntryForm can open ProjectForm in a modal — which can open ResourceForm — which can open EntryForm again. That's a closed loop."

"When I first wrote these with static imports, Next.js threw a hydration error: *'In HTML, form cannot be a descendant of form'*. The modals were rendering inside the parent form tag, and the circular imports were causing unpredictable component trees."

---

### 3:05 – 3:30 | The Fix

> *[Show the `dynamic()` import at the top of EntryForm.tsx]*

"I solved both problems at once using `next/dynamic`. By lazy-loading ProjectForm and ResourceForm inside EntryForm — and vice versa — the circular import chain is broken at build time. And because the modals are rendered outside the `<form>` tag using a Fragment wrapper, the nested form HTML error disappears entirely."

"This also has a nice side effect — the sub-forms only load their JavaScript bundle when the user actually clicks 'New Project', not on page load."

---

## PART 4 — Wrap-up (3:30 – 3:45)

> *[Switch back to dashboard]*

### 3:30 – 3:45 | Close

"That's DevLog — a full-stack journaling tool with type-safe validation end-to-end, real-time cache updates via React Query, and a component architecture that handles deep cross-linking between entities cleanly."

"Code is on GitHub, README has full setup instructions. Thanks for watching."

---

> ✅ **End recording** — target: ~3:45